EVAL HARNESS · RETRIEVAL · RECEIPTS

Bring *Receipts*, Not Vocabulary

Five hands-on drills to build the eval sets, ablations, and shipped repo that prove you can do the LLM engineering work — not just talk about it.

NAME

DATE

TARGET ROLE / REQ

1 Hand-Label a 30-Case Golden Dataset

- Pick **one real task** from your job — a router, a classifier, a Q&A bot — and hand-label **30 cases** yourself.
- Get a second person to label the same 30, *independently*, no peeking at your answers.
- List every disagreement, write **why** in one sentence each, then resolve it to a final label.
- Count your raw agreement rate before resolution — that's your `vibes-good` vs `actually-good` baseline.

AGREEMENT RATE

2 Run Your Own Chunking Ablation

- Take one document you'd retrieve from — a policy doc, a README, a spec.
- Split it two ways: **fixed 512-token chunks** vs *structure-aware* chunks that keep tables and lists intact.
- Write 10 questions whose answers span a table or a multi-step instruction.
- Run retrieval on both sets and record the **hit-rate** — the % where the right chunk actually surfaced.

HIT-RATE BY METHOD

3 Diagram One Agent's Failure Modes

- Sketch one agent workflow you've built or want to build — boxes and arrows, paper or a tool.
- Mark **where it retries** and **where it falls back** to a cheaper model.
- Write your target *p95 latency* and *per-run token cost* directly on the diagram.
- No real numbers yet? Write the ceiling you'd accept and what you'd measure to check it.

LATENCY & COST TARGETS

4 Try Context Before Fine-Tuning

- Pick a classification or routing problem you'd normally reach for **fine-tuning** to solve.
- Before any training loop, put the taxonomy plus **5–6 labeled examples** straight into the prompt's context window.
- Test it against *20 held-out cases* and record accuracy and time spent.
- One sentence: what would you have needed to see to justify the extra weeks?

CONTEXT VS FINE-TUNE

5 Ship the route_ticket.py Pattern on Your Own Data

- Adapt `route_ticket.py` to a task from your own job — classify **and** explain in one call.
- Build a `gold.jsonl` of **50 examples** you labeled by hand.
- Write an `evaluate()` that prints your own match rate against those labels.
- Push it public with a *README* — clone, run one command, watch it score itself. No notebook-only entries.

```
def evaluate(gold_path="gold.jsonl"):
```

REPO LINK + MATCH %